Selecting Reliable LLM Reasoning with Lightweight Process and Target Verification

DASE7506 MP2 Experimental Report

Fall 2026

Abstract

We study selection among four supplied reasoning traces for each grade-school math problem. Starting from answer agreement, text statistics and fixed embeddings, we train a four-way group-ranking model and add explicit checks of arithmetic steps, use of question conditions, answer-process linkage, and alignment with the quantity requested. The final role-aware process model obtains 743/798 (93.11%) on validation and 394/467 (84.37%) on the supplied test bank. Its eight final target/source features improve validation by one question over the preceding process model, but a five-fold train-only diagnostic decreases by one hard question. Because test outcomes were visible during later development and model selection, the test comparison is exploratory and is not an untouched estimate. We report that limitation explicitly alongside the reproducible checkpoint and resource costs.

1 Task, question, and protocol

For each question q, the supplied bank contains four candidate solution texts t_i and fixed 1,024-dimensional embeddings e_i. The selector returns one index $\hat{\imath} \in \{0, 1, 2, 3\}$. The official scorer checks only whether the selected candidate's parsed final numeric answer equals the reference; it does not grade every intermediate step. More than one candidate may be correct. The supplied splits contain 4,755 training, 798 validation, and 467 test questions, with no overlapping questions.

Our question is whether inspecting the calculation process and the target quantity helps beyond final-answer voting or whole-trace representations. The working hypothesis is that a candidate can perform locally valid arithmetic while answering the wrong quantity, such as original price instead of amount paid. We fit learned parameters only from the supplied training inputs and training references. Validation scores are development evidence. Test scores from later experiments are descriptive because earlier test results were already visible.

2 Method development

2.1 Baselines and learned ranking

Answer agreement clusters candidates by the official final-answer parser and selects a member of the largest cluster. The text model fits regularized logistic regression to 20 hand-built features including answer agreement, length and question overlap. The embedding model fits a ridge-regression score to standardized supplied embeddings. Pairwise comparison learns from correct–incorrect candidate pairs within a question. These provide independent checks on what information is useful: voting uses only answer equivalence, text features use surface and question cues, and embeddings use a frozen semantic representation.

The later group-choice models score all four candidates together. Let x_{gi} be the feature vector for candidate i in group g, $s_{gi} = w^\top(x_{gi} - \mu)/\sigma$, and C_g the set of correct candidates. We minimize the multi-correct group-softmax loss

$$\mathcal{L}(w) = \sum_g \left[\log \sum_{j=1}^{4} e^{s_{gj}} - \log \sum_{i \in C_g} e^{s_{gi}} \right] + \frac{\lambda}{2} \|w\|_2^2. \tag{1}$$

This objective gives credit when probability mass goes to any correct candidate, matching the task better than a separate binary decision on each trace. We use $\lambda = 25$, full-batch Adam for 300 epochs, learning rate 0.02, $\beta_1 = 0.9$, $\beta_2 = 0.999$, and $\epsilon = 10^{-8}$. Means and scales are learned from training inputs only; near-zero scales become one. The final fit starts at zero with no shuffling, so it has no random seed.

2.2 Reasoning and process features

The combined reasoning model adds four feature families to the group-choice base: arithmetic consistency of explicit equations; whether question numbers and conditions are mentioned or visibly used; where candidates first diverge in numerical/equation sequences; and whether the final answer is connected to late calculation results. The process-chain extension parses explicit equality chains with a restricted arithmetic evaluator, checks numerical equality to tolerance 10^{-3}, and links a verified result to a later step when it appears as an operand. Nine features summarize valid/invalid steps, reuse, graph depth, source-number coverage and final-answer linkage. The parser never executes candidate text as program code.

The selected model appends eight target/source features to this 1,082-dimensional process representation, making 1,090 features. It extracts the last question phrase beginning with a cue such as "how many" or "what is" and compares it with text near the candidate's final-answer marker. Features measure target keyword recall and overlap, target-role mentions, two narrow role conflicts, use of question percentages and currency values in verified equations, missing-source fraction, and target-unit mention. These are lexical and numeric proxies, not a proof of semantic correctness. Inference returns the highest of four linear scores; ties go to the lowest index. It uses only the passed bank, supplied embeddings and its checkpoint, never reference answers or ID-based lookup.

3 Experimental results

All rows use the same official scorer. The later test results are marked exploratory: testing earlier methods preceded development of the later feature sets. The table highlights representative stages rather than every experiment; the repository contains method-specific reports and the full experiment log.

Table 1: Official selection accuracy. Test values for later methods are exploratory.

Method	Validation / 798	Test / 467
Answer agreement	688 (86.22%)	323 (69.16%)
Text-feature logistic regression	719 (90.10%)	350 (74.95%)
Fixed-embedding ridge score	712 (89.22%)	360 (77.09%)
Pairwise candidate comparison	734 (91.98%)	378 (80.94%)
Group choice	742 (92.98%)	385 (82.44%)
Combined reasoning features	739 (92.61%)	389 (83.30%)
Process-chain verifier	742 (92.98%)	391 (83.73%)
+ Target-role/source features (selected)	**743 (93.11%)**	**394 (84.37%)**

The process-chain verifier is a controlled feature ablation over combined reasoning: it adds nine features with the same group-choice objective and optimizer. It gains three validation questions and two test questions. The final role/source extension adds eight features with the same training settings, gaining one validation question and three test questions over process-chain verification. On validation, 30 selected indices change; two errors become correct and one correct case becomes wrong. On test, nine indices change, with three becoming correct and none becoming wrong. These are small differences; they do not establish a robust advantage.

A five-fold train-only out-of-fold (OOF) diagnostic used seed 2026 for fold assignment. On 2,114 hard training groups, the process model selected 1,731 correctly; the role/source model selected 1,730. Overall OOF counts were 4,372/4,755 and 4,371/4,755 respectively. The role features were designed after examining OOF errors, so this is diagnostic rather than an independent holdout. It provides a useful counterweight to the small positive validation and test differences.

4 Error analysis, resources, and reproducibility

Process parsing recognizes explicit elementary calculations but misses implicit operations, algebraic shortcuts and many equivalent forms. A valid solution may express 25%as 25/100 rather than the recognized 0.25 factor. A candidate may mention a relevant number or unit without using it correctly. The narrow role-conflict vocabulary can also miss synonyms. Therefore the features are soft evidence learned by the ranker, not hard rejection rules. Parser coverage and individual failure modes are recorded in the corresponding method reports.

The final checkpoint is an NPZ containing three float32 arrays—means, scales, and weights—of length 1,090 each: 13,080 raw array bytes and 13,830 bytes on disk, below the 16 MiB asset limit. With four thread environment variables, an external wall-clock command on local Apple Silicon arm64 measured 3.24 s for one fresh test

prediction process plus independent scoring, including startup. The reference Xeon Platinum 8457C determines the course's 30 s eligibility; the local result alone cannot prove it. Python 3.12.14 and NumPy 2.5.3 were used. The exact commands, official JSON results, SHA-256 checkpoint hash, and code entry point are in `code/SUBMISSION.md`; no retraining is needed for scoring.

A major protocol limitation remains: previous test results were viewed before this final feature design, and the model was chosen partly because its test accuracy was highest. This conflicts with the assignment instruction to use validation, rather than test, for method selection. We disclose the chronology instead of presenting the test result as a clean holdout. A strictly compliant final choice would need independent validation-only selection or an instructor ruling. The selected model itself is frozen for reproduction.

5 Conclusion and disclosure

The experiments show substantial improvements from answer voting to learned candidate comparison and process-aware ranking on this bank. The last eight target/source cues produce only a one-question validation gain and a one-question decline in OOF hard groups, so their apparent test benefit should be treated cautiously. Future work would need stronger independent evaluation and more reliable semantic checks of quantity roles and dependency chains.

Codex AI assistance was used substantially for design discussion, implementation, experiment automation, analysis, and report preparation. The supplied dataset, embeddings, parser and evaluator were reused. No external training data, additional pretrained weights, or inference API was used. The student should review and understand the code and claims before submission.